

Chapter 1 Basics concepts of hydrological models: routing

Olivier Bonte

2026-07-01

To get more insight into the routing concepts explained in the theory, a computational guide is given here

First order linear reservoir

A linear reservoir with storage S [m³] and a constant input I [m³/s] is characterised by

$$S = KQ \quad (1)$$

with K [s] the residence time and Q [m³/s] the outflow. The mass balance equation is given by

$$\frac{dS}{dt} = K \frac{dQ}{dt} = I - Q \quad (2)$$

As this is a linear ordinary differential equation, it can be solved analytically using Laplace transforms (see your course in the 2nd bachelor year on [Differential Equations](#)). Remember that although the definition of a Laplace transform might seem complex

$$F(s) = \mathcal{L}\{f(t)\} = \int_0^\infty f(t)e^{-st}dt,$$

in practice you only need to apply a limited set of transforms to solve linear ODEs, which are summarised in the table below.

$f(t)$	$F(s) = \mathcal{L}\{f(t)\}$
$H(t)$	$\frac{1}{s}$

$f(t)$	$F(s) = \mathcal{L}\{f(t)\}$
t^n	$\frac{n!}{s^{n+1}}$
e^{at}	$\frac{1}{s-a}$
$f'(t)$	$sF(s) - f(0)$

where $H(t)$ is the Heaviside function, which is defined as $H(t) = 0$ for $t < 0$ and $H(t) = 1$ for $t \geq 0$. n is a positive integer and a is a real number. Applying the Laplace transform to equation Equation 2 gives with $Q(t=0) = Q_0 = S_0/K$:

$$K(sQ(s) - Q_0) = \frac{I}{s} - Q(s)$$

Isolating $Q(s)$ gives:

$$Q(s) = \frac{I}{s(1 + Ks)} + \frac{KQ_0}{1 + Ks}$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(Q(s)) = Q(t)$. For the term containing Q_0 :

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1 + Ks}\right) = \frac{K}{K}Q_0\mathcal{L}^{-1}\left(\frac{1}{1/K + s}\right) = Q_0e^{-t/K} \quad (3)$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1 + Ks)} = \frac{A}{s} + \frac{B}{1 + Ks}$$

To solve for A , multiply both sides by s and set $s = 0$:

$$\frac{I}{1 + Ks}\Big|_{s=0} = A \iff A = I$$

To solve for B , multiply both sides by $(1 + Ks)$ and set $s = -1/K$:

$$\frac{I}{s}\Big|_{s=-1/K} = B \iff B = -IK$$

We now need to apply the inverse Laplace transform to get $Q(t)$. Trivially, $\mathcal{L}^{-1}(Q(s)) = Q(t)$. For the term containing Q_0 :

$$\mathcal{L}^{-1}\left(\frac{KQ_0}{1+Ks}\right) = \frac{K}{K}Q_0\mathcal{L}^{-1}\left(\frac{1}{1/K+s}\right) = Q_0e^{-t/K} \quad (4)$$

The term with I is a bit more convolved, but can be solved using partial fraction decomposition:

$$\frac{I}{s(1+Ks)} = \frac{A}{s} + \frac{B}{1+Ks}$$

To solve for A , multiply both sides by s and set $s = 0$:

$$\frac{I}{1+Ks}\Big|_{s=0} = A \iff A = I$$

To solve for B , multiply both sides by $(1+Ks)$ and set $s = -1/K$:

$$\frac{I}{s}\Big|_{s=-1/K} = B \iff B = -IK$$

Now we can apply the inverse laplace transform on the partial fraction decomposition:

$$\mathcal{L}^{-1}\left(\frac{I}{s} - \frac{IK}{1+Ks}\right) = \mathcal{L}^{-1}\left(\frac{I}{s} - \frac{I}{s+1/K}\right) = I - Ie^{-t/K} = I(1 - e^{-t/K}) \quad (5)$$

Combining Equation 4 and Equation 5 gives the final solution for $Q(t)$:

$$Q(t) = I(1 - e^{-t/K}) + Q_0e^{-t/K}$$

E.g. for $Q_0 = 0.1 \text{ m}^3/\text{h}$ and $K = 7 \text{ h}$, we can plot the outflow $Q(t)$ for different constant inflows I . In the examples that follow, we'll use m^3/h and h for the units instead of the standard SI-units for convenience. Define the analytical solutions as a function:

```
import matplotlib.pyplot as plt
import numpy as np

def Q_analytical_linear(t, I, K, Q0):
    return I * (1 - np.exp(-t / K)) + Q0 * np.exp(-t / K)
```

define the parameters and the three different inflows:

```

Q_0 = 0.1 # m3/h
K = 7 # h
t = np.linspace(0, 40, 200) # h
I = [0.0, 0.06, 0.12] # m3/h

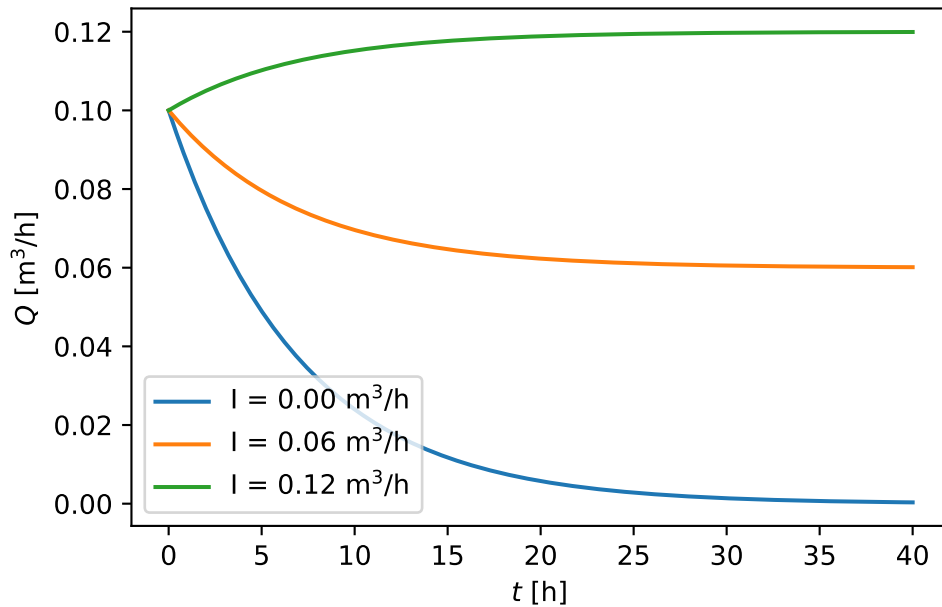
```

to then plot the results:

```

fig, ax = plt.subplots()
for I_ in I:
    Q_ = Q_analytical_linear(t, I_, K, Q_0)
    ax.plot(t, Q_, label=rf"I = {I_:.2f} m3/h")
ax.set_xlabel(r"$t$ [h]")
ax.set_ylabel(r"$Q$ [m3/h]")
ax.legend()

```



Additionally, note that we can automate the algebraic routines above using [SymPy](#).